

OOPJ — Day 11

Saturday, 19 September 2026 | File I/O Recap, transient, Collections & Generics

1. File I/O and Serialization Recap

- `ObjectOutputStream` writes objects to file; `ObjectInputStream` reads them back.
- `readObject()` deserializes data from file, reconstructs the object, and returns it as an `Object` reference.
 - Must downcast to the actual type before use.
- Standard pattern: loop with `while(true)` + `EOFException` to read until end of file.

```
try {
    int index = 0;
    while (true) {
        Object data = objectStream.readObject();
        manager[index] = (Manager) data; // downcast to known type
        index++;
    }
} catch (EOFException e) {
    // File fully read — expected, not an error
    numOfBand = index; // IMPORTANT: update count after load
} catch (ClassNotFoundException e) {
    e.printStackTrace();
}
```

Bug fixed live: after loading from file, display showed nothing.

- Root cause: `numOfBand` never updated after the load loop completed.
- Fix: assign `numOfBand = index` before exiting the try block.
- Described as a “temporary fix, not a proper solution” — a proper design would track size automatically (as `ArrayList` does).

Second bug: array typed as `Manager[]` instead of `Employee[]`.

- `readObject()` returned `Employee` subtypes; casting to `Manager` failed for Engineer/Salesperson entries.
- Correct approach: declare array as `Employee[]`; downcast to specific subtype when needed.
- If typed as `Employee`, any subtype (Manager, Engineer, Salesperson) loads correctly.

2. transient Keyword and Serialization Control

- By default, all fields of a serializable object are written to the media (file).
- `transient` keyword marks a field to be skipped during serialization.
 - Field value is not stored; when deserialized, it takes its default value.

```
class Student implements Serializable {
    String name;
    String address;
    transient boolean gender; // will NOT be written to file
    transient int age;        // will NOT be written to file
}
```

- Demonstrated live with `student.tht` file:
 - Before adding `transient`: name, address, gender, age all visible in file.
 - After adding `transient` to gender and age: only name and address appear in file.
- Use case: sensitive or derived fields that should not be persisted (e.g., passwords, computed values, gender).

3. Arrays vs. Collections: Core Comparison

Six drawbacks of arrays that motivate the use of collections:

--	--	--

#	Drawback	Detail
1	Fixed size	Cannot grow; always stays at declared dimension.
2	Cannot grow on demand	If 10 declared and 11th needed, impossible without manual recreation.
3	Predefined size	Must know size upfront at declaration time.
4	Memory wastage	Abhay's three arrays of size 40 held only 2 elements; 38 slots wasted.
5	Not built on a standard data structure	No underlying linked list, queue, stack, etc.
6	No supporting methods	Cannot insert-at-position, remove-from-position, search, sort natively.

ARRAY DECLARATIONS FOR REFERENCE

```
int[] arr = new int[10];           // primitive array, fixed at 10
Employee[] empArr = new Employee[10]; // object array, fixed at 10
```

- Collections address all six drawbacks:
 - Grow on demand: no predefined size, no memory wastage.
 - Hold heterogeneous or homogeneous data.
 - Built on standard data structures (linked list, array, tree, etc.).
 - Come with rich supporting methods (`add` , `remove` , `contains` , `iterator` , `sort` , `search`).
- When to use which — requirement-driven decision:
 - Frequent retrieval / random access: prefer `ArrayList` (array-backed, O(1) index access).
 - Frequent insert/delete: prefer `LinkedList` (pointer rewiring, no array copy).
 - Multithreaded access to same object: prefer `Vector` (all methods synchronized).
- Performance note on arrays vs. linked structures:
 - Array: accessing 80th element out of 100 → `arr[79]` , direct memory address, O(1).
 - LinkedList: accessing 80th element → must traverse node by node from start, O(n).

4. Java Collection Framework: Key Interfaces

- The Collection Framework is a series of interfaces and classes for storing, manipulating, and traversing groups of objects.
 - Not a new concept: C++ had STL (Standard Template Library); Java extended and formalized it.
 - Introduced from Java 1.2 onward (some legacy classes from Java 1.0).
- Nine primary interfaces in the framework:
 1. Collection (root interface)
 2. List
 3. Set
 4. SortedSet
 5. NavigableSet (introduced Java 1.6+)
 6. Queue
 7. Map
 8. SortedMap
 9. NavigableMap
- `Map` does not inherit from `Collection` but is still part of the Collection Framework.
 - It does not fit the “group of objects as single entity” model; it stores key-value pairs.
- No concrete class directly implements `Collection` .
 - Concrete classes implement sub-interfaces (List, Set, Queue) which inherit from Collection.
 - `Collection` serves purely as the root contract.

`Collection` (interface) vs. `Collections` (utility class) — a common interview trap:

	Collection	Collections
Type	Interface	Class
Purpose	Represent a group of objects as a single entity	Utility methods for operating on collections
Key methods	<code>add</code> , <code>remove</code> , <code>contains</code> , <code>isEmpty</code> , <code>iterator</code> , <code>size</code>	<code>sort</code> , <code>search</code> , <code>shuffle</code> , <code>reverse</code> , etc.

5. Cursors: Enumeration, Iterator, ListIterator

- A **cursor** is an interface used on a collection to retrieve elements one by one.
- Three cursor types provided in the Java Collection Framework:

Cursor	Also Known As	Direction	Applicable To	Operations	Obtained Via
Enumeration	Legacy cursor	Forward only	Legacy classes only (Vector, Stack)	Read only	<code>.elements()</code>
Iterator	Universal cursor	Forward only	All collection classes (except legacy)	Read, Remove	<code>.iterator()</code>
ListIterator	—	Bidirectional	List classes only	Read, Remove, Add, Replace	<code>.listIterator()</code>

Enumeration (Legacy Cursor)

- Introduced in Java 1.0, predates the Collection Framework.
- Only applicable to legacy classes: `Vector`, `Stack`, `Hashtable`.
- Two methods only:
 - `hasMoreElements()` — checks if more elements remain.
 - `nextElement()` — fetches the next element.

```
Vector<String> v = new Vector<>();
v.add("Alpha");
v.add("Beta");

Enumeration<String> e = v.elements();
while (e.hasMoreElements()) {
    System.out.println(e.nextElement());
}
```

Iterator (Universal Cursor)

- Works with all Collection classes except legacy classes.
- Three primary methods:
 - `hasNext()` — checks if next element exists.
 - `next()` — fetches and returns the next element.
 - `remove()` — removes the current element from the collection.

```
List<String> list = new ArrayList<>();
list.add("First");
list.add("Second");
list.add("Third");

Iterator<String> iter = list.iterator();
while (iter.hasNext()) {
    String data = iter.next();
    System.out.println(data);
    // iter.remove(); // optionally remove current element
}
```

- Three drawbacks of Iterator:
 1. Forward only — cannot traverse backwards (no `previous()`).
 2. No replace — cannot update/set an element in place.
 3. No add — cannot insert new elements during traversal.
- Java 1.8+ added `forEachRemaining()` via Consumer/lambda, but `hasNext` / `next` / `remove` remain the primary methods.

ListIterator (Bidirectional Cursor)

- Bidirectional: can traverse forward (`hasNext` / `next`) and backward (`hasPrevious` / `previous`).
- Only available for classes implementing the `List` interface (`ArrayList`, `LinkedList`, `Vector`).
- Cannot be obtained on `Set` or `Map`.
- Nine methods total:
 - `hasNext()`, `next()`, `nextIndex()`
 - `hasPrevious()`, `previous()`, `previousIndex()`
 - `remove()`, `add()`, `set()` (replace)

```

List<String> list = new ArrayList<>();
list.add("First");
list.add("Second");
list.add("Third");

ListIterator<String> li = list.listIterator();

// Forward traversal
System.out.println("Forward:");
while (li.hasNext()) {
    System.out.println(li.next());
}

// Backward traversal
System.out.println("Backward:");
while (li.hasPrevious()) {
    System.out.println(li.previous());
}

```

- Cursor positioning:
 - Before any `next()` call, cursor sits at index -1 (before first element).
 - `next()` retrieves element at index 0, then 1, etc.
 - For `previous()`, cursor starts at max index (after last element).
 - Calling `previous()` without first calling `next()` throws `NoSuchElementException`.
- Critical rule: always guard `next()` / `previous()` with `hasNext()` / `hasPrevious()`.

```

// WRONG — will throw NoSuchElementException at end of list
while (li.next() != null) {
    System.out.println(li.next()); // skips elements AND crashes at end
}

// RIGHT
while (li.hasNext()) {
    System.out.println(li.next());
}

```

6. List Interface and Implementations

- `List` inherits from `Collection`; introduced in Java 1.2.
- Characteristics shared by all List implementations:
 - Duplicates allowed — same value can appear multiple times.
 - Insertion order preserved — elements retrieved in the order they were added.
 - Null values allowed.
 - Heterogeneous or homogeneous data (without generics).

```

List list = new ArrayList();
list.add("First");
list.add("First"); // duplicate — allowed
list.add("First"); // allowed again
list.add(null);    // null — allowed

```

ArrayList

- Implements `List` using a resizable (growable) internal array.
- Also implements: `RandomAccess`, `Cloneable`, `Serializable`.
- `RandomAccess` means any element can be accessed directly by index in O(1).

```

List<String> list = new ArrayList<>(); // default initial capacity = 10
list.add("First");
list.add("Second");
list.add("Third");

System.out.println(list.size()); // 3
System.out.println(list.get(1)); // "Second" — random access by index

```

- Internal growth mechanism:
 - When capacity is exhausted (e.g., 10 elements filled), `ArrayList` creates a new internal array (roughly 1.5× or 2× size).
 - Copies all existing elements into the new array.
 - This copy operation is the performance cost of `ArrayList` growth.
- Two key performance drawbacks:
 1. Growth: creating and copying to a new array on overflow degrades performance.
 2. Deletion: removing the 5th element from a 10-element list requires creating a new array, copying elements 1-4, skipping 5, copying 6-10.
- Best used when: frequent retrieval/traversal, infrequent insert/delete.

LinkedList

- Implements `List` using a doubly linked list (each node holds data, a reference to previous, and a reference to next).
- Also implements: `Cloneable`, `Serializable`, `Deque`.
- Does NOT implement `RandomAccess` — cannot jump to an index directly.

```
List<String> list = new LinkedList<>();
list.add("First");
list.add("Second");
list.add("Third");
list.add(1, "New Data"); // insert at index 1 — efficient, just rewires pointers

ListIterator<String> li = list.listIterator();
while (li.hasNext()) {
    System.out.println(li.next());
}
```

INTERNAL NODE STRUCTURE (CONCEPTUALLY)

```
class Node<E> {
    E data;
    Node<E> previous;
    Node<E> next;
}
```

- Insertion/deletion is $O(1)$ once position is known — just rewire previous/next pointers, no array copy.
- Traversal to a specific index is $O(n)$ — must walk node by node from start or end.
- Best used when: frequent insert/delete in the middle; worst when: frequent random access or traversal.

Vector (Legacy)

- Functionally identical to `ArrayList` (resizable array, same interfaces: `List`, `RandomAccess`, `Cloneable`, `Serializable`).
- Key difference: all methods are synchronized.
 - Only one thread can execute a `Vector` method at a time.
 - Guarantees data consistency in multithreaded applications.
- `ArrayList` is not thread-safe; if two threads modify the same `ArrayList` simultaneously, data inconsistency results.
- `Vector` is the correct choice when multiple threads act on the same collection object.
- Introduced in Java 1.0; later retrofitted into the Collection Framework.

```
// ArrayList — NOT thread-safe
List<String> arrayList = new ArrayList<>();

// Vector — thread-safe (synchronized methods)
List<String> vector = new Vector<>();
```

`Vector` also inherits into `Stack` (LIFO — last in, first out).

7. Generics

- Problem without generics: `ArrayList` accepts any `Object`, so heterogeneous data can be mixed accidentally.

```
// Without generics — no type safety
List list = new ArrayList();
list.add("Hello");
```

```
list.add(42);           // integer added — no compile error
list.add(new Employee()); // employee added — no compile error

Iterator iter = list.iterator();
while (iter.hasNext()) {
    String s = (String) iter.next(); // ClassCastException when it hits Integer!
}
```

- Generics (Java's equivalent of C++ templates) solve this with compile-time type checking.
 - Declare a type placeholder (`<E>` , `<T>` , or any name) at class definition.
 - Actual type is bound when the object is created.
 - All methods in the class then operate on that specific type.

```
// With generics — type-safe
List<String> list = new ArrayList<String>();
list.add("First");
list.add("Second");
// list.add(42); // COMPILER ERROR — only String allowed

Iterator<String> iter = list.iterator();
while (iter.hasNext()) {
    String data = iter.next(); // no downcast needed
    System.out.println(data);
}
```

How Generics Work Internally

```
// Conceptual generic class (similar to how ArrayList is defined)
class MyList<E> {
    E data;

    public void add(E value) {
        this.data = value;
    }

    public E get() {
        return data;
    }
}

// Usage — type bound at object creation
MyList<String> strList = new MyList<String>();
strList.add("Hello");
String val = strList.get(); // no cast needed

MyList<Integer> intList = new MyList<Integer>();
intList.add(100);
Integer num = intList.get();
```

Multiple Generic Type Parameters

```
// Two type parameters <T, U>
class Data<T, U> {
    T value1;
    U value2;

    public T getValue1() { return value1; }
    public U getValue2() { return value2; }
}

// First object: T = Integer, U = String
Data<Integer, String> obj1 = new Data<>();
obj1.value1 = 77;
obj1.value2 = "Hello";

// Second object: T = String, U = Integer
Data<String, Integer> obj2 = new Data<>();
obj2.value1 = "World";
obj2.value2 = 42;
```

`Map` uses exactly this pattern: `Map<K, V>` where K = key type, V = value type.

Type Inference (Diamond Operator)

```
// Before type inference (Java 6 and earlier)
List<String> list = new ArrayList<String>();

// With type inference (Java 7+) — compiler infers type from LHS
List<String> list = new ArrayList<>(); // <> is the "diamond operator"
List<Integer> intList = new ArrayList<>();
```

The compiler detects the generic type from the left-hand side declaration automatically. No need to repeat the type on the right-hand side.

Key Rules and Limitations

- Java generics cannot use primitive types — must use wrapper classes.

```
List<int> list = new ArrayList<>(); // COMPILER ERROR
List<Integer> list = new ArrayList<>(); // CORRECT — uses wrapper class
```

Boxing (auto-boxing) handles primitive-to-wrapper conversion automatically:

```
List<Integer> list = new ArrayList<>();
list.add(11); // auto-boxing: int 11 → Integer(11)
```

C++ templates allow primitive types; Java generics do not — this is the primary difference.

Handling Heterogeneous Data with instanceof

```
List list = new ArrayList(); // raw type, heterogeneous
list.add("First");
list.add("Second");
list.add(11); // auto-boxed to Integer

Iterator iter = list.iterator();
while (iter.hasNext()) {
    Object data = iter.next();
    if (data instanceof String str) {
        System.out.println("String: " + str);
    } else if (data instanceof Integer obj) {
        System.out.println("Integer: " + obj);
    }
}
```

8. Live Demo: Employee App Refactored with ArrayList

- Original code used three fixed-size arrays (`Manager[40]` , `Engineer[40]` , `Salesperson[40]`).
 - Only 2 elements added; 38 slots wasted per array.
 - Manual index tracking required (`numOfBand` , etc.).
- Refactored to use a single `ArrayList<Employee>` — all subtypes stored polymorphically.

```
// Old approach — fixed arrays, manual index tracking
Manager[] managers = new Manager[40];
Engineer[] engineers = new Engineer[40];
// ... manual numOfManagers, numOfEngineers counters

// New approach — single dynamic collection
List<Employee> allEmployees = new ArrayList<>();

// Adding employees
Manager objManager = new Manager();
// ... populate objManager fields
allEmployees.add(objManager); // upcasting to Employee automatically

Engineer objEngineer = new Engineer();
// ... populate objEngineer fields
allEmployees.add(objEngineer);
```

TRAVERSAL USING ITERATOR WITH INSTANCEOF CHECKS TO HANDLE SUBTYPES

```
Iterator<Employee> itr = allEmployees.iterator();
while (itr.hasNext()) {
    Employee objEmployee = itr.next();
    System.out.println(objEmployee); // uses toString() auto-generated by IDE
}
```

DISPLAYING ONLY MANAGERS — TWO APPROACHES DISCUSSED

```
// Approach 1: instanceof check
Iterator<Employee> itr = allEmployees.iterator();
while (itr.hasNext()) {
    Employee obj = itr.next();
    if (obj instanceof Manager) {
        Manager m = (Manager) obj;
        System.out.println(m);
    }
}

// Approach 2: Pattern matching (modern Java)
while (itr.hasNext()) {
    Employee obj = itr.next();
    if (obj instanceof Manager m) {
        System.out.println(m);
    } else if (obj instanceof Engineer e) {
        System.out.println(e);
    } else if (obj instanceof Salesperson s) {
        System.out.println(s);
    }
}
```

- Key benefits of the refactored approach:
 - No manual index tracking — ArrayList manages size internally.
 - No fixed-size limitation — grows as employees are added.
 - No memory wastage — only as many slots as needed.
 - Single collection for all employee types via polymorphism.
- Random access still possible via `get(index)` since ArrayList implements RandomAccess:

```
Employee e = allEmployees.get(4); // direct index access — O(1)
// Cannot use allEmployees[4] — square bracket syntax only works for raw arrays
```

9. Next Steps and Exercises

- **Refactor employee application to use ArrayList instead of arrays.**
- **Complete the six ArrayList exercises shared by Pankaj.**
- **Complete the encryption and decryption assignment.**
- Next lecture: Sunday 20th September, 9 AM to 1 PM, continuing with Collection Framework (Set, Map).
- From Monday onwards, sessions resume at 4:30 PM.
- Day 11 notes and Day 11 code ZIP shared at end of session — verify both files downloaded correctly.
- If a shared file does not open, DM the instructor or get it from a classmate.

Meeting transcript reference: notes.granola.ai/t/a7b25d54...